Estándar de Arquitectura

.NET 10 + Angular (v20+)

Documento de referencia obligatorio para el desarrollo de proyectos full stack

1. Introducción y Alcance

Este documento define la arquitectura y los estándares técnicos que deben seguirse al construir proyectos full stack, tanto en el backend (.NET) como en el frontend (Angular).

El backend sigue los principios de Arquitectura Limpia (Clean Architecture) con el patrón CQRS. El frontend sigue el Estándar de Desarrollo Frontend Angular. Ambos estándares son de cumplimiento obligatorio.

2. Arquitectura Backend (.NET)

El backend debe construirse siguiendo Arquitectura Limpia (también conocida como Arquitectura Hexagonal o de Puertos y Adaptadores), combinada con el patrón CQRS (Command Query Responsibility Segregation) para separar la lógica de lectura de la de escritura.

La regla de dependencia es estricta: las capas exteriores dependen de las interiores, pero las interiores no conocen nada de las exteriores.

```
API  ->  Application  ->  Domain
Infrastructure  ->  Application  ->  Domain
```

2.1. Domain

Capa más interna. Contiene las entidades de negocio (ej. Producto, Categoria) y las reglas fundamentales del dominio. No depende de ninguna otra capa ni conoce frameworks o tecnologías externas.

2.2. Application

Orquesta los casos de uso. Aquí se definen:

- **Commands y Queries (CQRS):** objetos que representan la intención de modificar o leer datos.
- **Handlers:** clases que procesan cada Command o Query.
- **Puertos (interfaces):** contratos como IProductoRepository, que definen qué se necesita del exterior sin saber cómo se implementa (Principio de Inversión de Dependencias).

2.3. Infrastructure

Implementa los puertos definidos en Application. Contiene el acceso a datos real:

- **PersistenceContext:** DbContext de Entity Framework Core, mapea entidades a tablas de SQL Server.

- **Repositorios concretos:** implementación real de las interfaces de repositorio definidas en Application.
- **Seeds:** clases para poblar datos iniciales.

2.4. API

Capa de presentación y punto de entrada. Expone la funcionalidad vía endpoints REST:

- **Program.cs:** configuración de inyección de dependencias, middlewares y pipeline HTTP.
- **Controllers:** reciben la petición HTTP, la delegan a Application vía MediatR o Wolverine, y devuelven la respuesta.

3. Arquitectura Frontend (Angular)

El frontend debe construirse en Angular (v20+) siguiendo el Estándar de Desarrollo Frontend del equipo AGAVAL. Estos lineamientos son de cumplimiento obligatorio.

3.1. Estructura del proyecto

```
src/
  app/
    core/            (funcionalidades globales)
      authentication/
      guards/
      interceptors/
      services/
    shared/          (elementos reutilizables entre features)
      components/
      pipes/
      directives/
    features/        (una carpeta por funcionalidad)
      nombre-feature/
        components/
        services/
        models/
        routes.ts
    app.config.ts
    app.routes.ts
    main.ts
```

Cada componente debe manejarse con separación de responsabilidades por archivo, como mínimo: nombre-componente.ts, .html, .scss y .spec.ts.

3.2. Angular moderno y rendimiento (obligatorio)

- **Zoneless:** uso obligatorio de provideZonelessChangeDetection(), sin dependencia de zone.js.
- **Configuración global:** providers en app.config.ts, con provideRouter() y provideHttpClient() nativos.
- **Signals:** obligatorio para el manejo de estado y la reactividad.
- **Control flow moderno:** usar @if, @for y @switch (no *ngIf / *ngFor).
- **Rendimiento:** todos los componentes con ChangeDetectionStrategy.OnPush.
- **Carga diferida:** lazy loading obligatorio para rutas y features.

3.3. Tipado, HTML y estilos

- **Tipado estricto:** modelos, respuestas HTTP y formularios tipados (sin any).
- **HTML semántico:** uso de etiquetas significativas (nav, article, header, etc.).
- **SCSS + BEM:** metodología BEM estricta para el nombrado de clases (ej. .producto-card__titulo).
- **Angular Material:** biblioteca principal de componentes UI del proyecto.
- **Personalización:** mediante tokens/variables o temas de Angular Material, nunca ::ng-deep ni !important.

3.4. Anti-patrones prohibidos

Las siguientes prácticas se consideran incumplimiento del estándar:

- Uso de any
- Lógica de negocio dentro del HTML
- Componentes con demasiadas responsabilidades
- Uso de !important o ::ng-deep
- Estilos inline en HTML
- Templates con lógica compleja o funciones pesadas ejecutadas en el template
- Uso de NgModules (el proyecto debe ser 100% standalone)

4. Flujo de una Solicitud Full Stack (ejemplo)

Para ilustrar cómo deben integrarse ambas arquitecturas, se describe el flujo completo para crear un nuevo registro desde la interfaz (ejemplo genérico, aplicable a cualquier entidad):

1. Interfaz (Angular): el usuario completa un formulario en un componente standalone (OnPush) y presiona "Guardar".
2. Servicio (Angular): un servicio inyectado con inject(HttpClient) envía un POST a la API con el DTO tipado.
3. Controlador (API): el Controller recibe la petición, crea un Command y lo envía vía MediatR.
4. Handler (Application): el CommandHandler valida los datos y usa el puerto (interfaz de repositorio) para persistir.
5. Inyección de dependencias: Program.cs resuelve la interfaz con su implementación concreta (Infrastructure).
6. Repositorio (Infrastructure): usa PersistenceContext (EF Core) para insertar el registro en SQL Server.
7. Respuesta: el registro creado regresa Infrastructure -> Application -> API, que responde 201 Created.
8. Actualización de UI (Angular): el componente actualiza un signal con los nuevos datos; @for repinta la vista automáticamente.

Este diseño asegura que la lógica de negocio no se vea afectada por cambios de tecnología, y que backend y frontend evolucionen de forma desacoplada pero consistente.